

GUIDE TECHNIQUE · AUTOMATISATION IA

v2.0 — Juin 2026

NotebookLM Multi-Output Pipeline

Architecture complète pour la génération automatisée
de tous les formats de sortie NotebookLM

Christian Levannier · Mandataire Indépendant
Automatisation & Veille IA · 4 juin 2026

Sommaire

1. Introduction & Vision d'ensemble	2
2. Prérequis & Environnement	2
3. Architecture du Pipeline v2.0	3
4. Configuration — pipeline_config.json v2	3
5. Phase 1 — Routeur de types	4
6. Phase 2 — Handlers de téléchargement par type	5
7. Tableau complet des formats de sortie	6
8. Phase 3 — Gestion des erreurs	7
9. Cas d'usage concrets (12 exemples)	7
10. Ordre d'implémentation recommandé	9
11. Post-processeurs optionnels	10
12. Dépannage — Erreurs connues & solutions	10
13. Artifact Configurateur v2 — Interface Cowork	11
14. Ressources & Références	14

1. Introduction & Vision d'ensemble

Ce guide documente l'architecture et l'implémentation du pipeline NotebookLM Multi-Output v2.0 — une extension du pipeline v1 (infographie uniquement) vers la génération automatisée de **tous les formats de sortie** proposés par NotebookLM Studio.

Le pipeline v1 permettait de générer des infographies depuis un notebook NotebookLM via automation Chrome (CDP). La v2.0 étend cette capacité à 9 types de sorties : audio, vidéo, slide deck, mind map, flashcards, quiz, rapport, data table et infographie.

L'objectif central est de disposer d'un **pipeline configurable** qui, à partir d'un seul fichier `pipeline_config.json`, orchestre la création parallèle ou séquentielle de tous les artefacts souhaités, gère les téléchargements selon la méthode adaptée à chaque type, et organise les sorties dans un dossier daté.

❑ **Certitude élevée** : Les types text/JSON (report, mind_map, quiz, flashcards, data_table) sont documentés et fonctionnels via MCP `download_artifact`. L'infographie est 100% validée (pipeline v1). Audio/vidéo/slide_deck sont à valider (streaming non testé sur Windows).

2. Prérequis & Environnement

Composant	Version / Détail	Statut
Python	3.10+	❑ Requis
MCP NotebookLM (studio_create, download_artifact, poll_artifact)	Version courante	❑ Installé
Chrome + CDP (websocket Python)	Chrome stable, port 9222	❑ Validé v1
httpx	pip install httpx	❑ Validé v1
Compte NotebookLM	Compte Google actif avec Studio activé	❑ Requis
Notebook existant	ID de notebook valide avec sources	❑ Requis
Cookies Chrome authentifiés	Session Google active dans Chrome	❑ Validé v1
WeasyPrint (optionnel)	Pour post-processing PDF des rapports Markdown	❑ Optionnel

⚠ **Windows spécifique** : Le bug du chemin Windows (backslash) est connu depuis la v1. Toujours utiliser `pathlib.Path` ou des chemins raw strings `r"C:\..."`. Le patch MCP pour les chemins est déjà appliqué.

3. Architecture du Pipeline v2.0

```
pipeline_config.json
└─ output_types: ["audio", "infographic", "quiz", ...]
    ↓
[ROUTEUR] — pour chaque type :
    ↓ studio_create(notebook_id, type, params)
    ↓ artifact_id ← stocké dans artifacts_map
    ↓ poll_artifact(artifact_id) × N → status: completed
    ↓ download via handler adapté au type
    ↓
outputs/{date}_{notebook}_{type}.{ext}
```

Le pipeline se compose de 4 composants principaux :

- 1

Config Loader — Lit `pipeline_config.json`, valide les types demandés, prépare les paramètres spécifiques à chaque type.
- 2

TYPE_ROUTER — Pour chaque type actif, construit l'appel `studio_create` avec les bons paramètres et stocke les `artifact_id` retournés.

3 Poll Manager — Surveille le statut de chaque artifact (polling toutes les 10s, timeout 10min, retry ×3 sur erreur).

4 Download Handler — Sélectionne et exécute la méthode de téléchargement adaptée : MCP `download_artifact` (texte/JSON/binaire) ou httpx+CDP (infographie).

4. Configuration — `pipeline_config.json` v2

Schéma complet du fichier de configuration. Seul `output_types` est obligatoire — tous les autres paramètres ont des valeurs par défaut.

```
{
  "notebook_id": "votre-notebook-id",
  "output_types": ["infographic", "audio", "quiz"],
  "outputs": {
    "infographic": {
      "orientation": "square",
      "style": "clay",
      "language": "fr"
    },
    "audio": {
      "format": "deep_dive",
      "length": "default",
      "language": "fr"
    },
    "video": {
      "format": "explainer",
      "style": "auto_select"
    },
    "slide_deck": {
      "format": "detailed_deck",
      "file_format": "pdf"
    },
    "mind_map": {
      "title": "Synthèse"
    },
    "flashcards": {
      "difficulty": "medium",
      "output_format": "html"
    },
    "quiz": {
      "question_count": 10,
      "difficulty": "medium",
      "output_format": "html"
    },
    "report": {
      "format": "Briefing Doc"
    },
    "data_table": {
      "description": "Tableau synthétique des données clés"
    }
  },
  "execution": {
```

```

"mode": "sequential",
"continue_on_error": true,
"polling_interval_s": 10,
"timeout_min": 10,
"retry_count": 3
},
"output_dir": "outputs"
}

```

Mode sequential vs parallel : Le mode `sequential` est recommandé pour éviter les rate limits de NotebookLM. Le mode `parallel` peut être utilisé pour les types légers (quiz, flashcards, report) avec un délai de 5s entre chaque appel `studio_create`.

5. Phase 1 — Routeur de types

Le routeur construit dynamiquement les paramètres de `studio_create` selon le type demandé. Chaque type expose ses propres paramètres spécifiques.

Paramètres `studio_create` par type

Type	Paramètres spécifiques	Valeurs possibles
<code>infographic</code>	orientation, detail_level, infographic_style	square/portrait/landscape · clay/flat/3d
<code>audio</code>	audio_format, audio_length	deep_dive/overview · default/short/long
<code>video</code>	video_format, visual_style	explainer/cinematic · auto_select
<code>slide_deck</code>	slide_format, slide_length	detailed_deck/overview · default
<code>mind_map</code>	title	chaîne libre
<code>flashcards</code>	difficulty	easy/medium/hard
<code>quiz</code>	question_count, difficulty	5-20 · easy/medium/hard
<code>report</code>	report_format	Briefing Doc · Study Guide · FAQ · Timeline
<code>data_table</code>	description	chaîne libre (instructions pour NLM)

Code du routeur principal

```

def build_studio_params(output_type: str, config: dict) -> dict:
    """Construit les paramètres studio_create pour un type donné."""
    type_config = config.get("outputs", {}).get(output_type, {})
    base = {"notebook_id": config["notebook_id"], "output_type": output_type}

    if output_type == "infographic":
        base.update({
            "orientation": type_config.get("orientation", "square"),
            "infographic_style": type_config.get("style", "clay"),

```

```

    })
    elif output_type == "audio":
        base.update({
            "audio_format": type_config.get("format", "deep_dive"),
            "audio_length": type_config.get("length", "default"),
        })
    elif output_type == "quiz":
        base.update({
            "question_count": type_config.get("question_count", 10),
            "difficulty": type_config.get("difficulty", "medium"),
        })
    elif output_type == "report":
        base.update({
            "report_format": type_config.get("format", "Briefing Doc"),
        })
    # ... autres types analogues

    return base

```

6. Phase 2 — Handlers de téléchargement

Deux familles de handlers selon le type d'artefact :

Handler A — MCP download_artifact (recommandé)

Fonctionne pour : **report, mind_map, flashcards, quiz, data_table, slide_deck, audio, video**
 Statut : documenté MCP · audio/vidéo/slide_deck à valider sur Windows (streaming binaire)

```

async def download_via_mcp(artifact_id: str, output_type: str,
                           output_dir: str, notebook_id: str) -> str:
    ext = EXT_MAP[output_type]          # ex: ".m4a", ".pdf", ".html"
    filename = f"{DATE}_{notebook_id}_{output_type}{ext}"
    filepath = Path(output_dir) / filename

    result = await mcp.download_artifact(
        artifact_id=artifact_id,
        output_path=str(filepath)
    )
    return str(filepath) if result.success else None

```

Handler B — httpx + CDP Cookies (infographie)

Requis pour : **infographic** (URL lh3.googleusercontent.com)
 Particularité Windows : suppression du cookie OSID obligatoire (bug Chrome/CDP Windows identifié en v1)

```

async def download_infographic_cdp(artifact_url: str,
                                   output_dir: str, notebook_id: str) -> str:
    # 1. Extraire les cookies depuis Chrome via CDP
    cookies = await extract_chrome_cookies()

    # ⚠ BUG WINDOWS : supprimer OSID pour éviter 403

```

```

cookies = {k: v for k, v in cookies.items() if k != "OSID"}

# 2. Télécharger avec httpx
filename = f"{DATE}_{notebook_id}_infographic.png"
filepath = Path(output_dir) / filename

async with httpx.AsyncClient() as client:
    response = await client.get(artifact_url, cookies=cookies)
    response.raise_for_status()
    filepath.write_bytes(response.content)

return str(filepath)

```

7. Tableau complet des formats de sortie

Type	Extension	Handler	Certitude	Notes
infographic	.png	httpx + CDP	100%	Validé pipeline v1 · Bug OSID Windows connu
report	.md	MCP download_artifact	95%	Sync · Documenté MCP
mind_map	.json + .html	MCP download_artifact	95%	Post-proc D3.js optionnel
flashcards	.html / .json	MCP download_artifact	95%	Sync · Prêt à ouvrir en navigateur
quiz	.html / .json	MCP download_artifact	95%	Sync · Prêt à ouvrir en navigateur
data_table	.csv	MCP download_artifact	95%	Sync · Import Excel/Sheets direct
audio	.m4a	MCP download_artifact	70%	Streaming binaire — à tester Windows
video	.mp4	MCP download_artifact	70%	Streaming binaire — à tester Windows
slide_deck	.pdf / .pptx	MCP download_artifact	75%	PDF via lh3.googleusercontent.com possible (même problème qu'infographic)

⚠ Risque principal identifié : Le téléchargement audio/vidéo via MCP n'est pas encore testé sur Windows avec streaming binaire. Si **download_artifact** échoue, fallback possible : handler httpx+CDP avec URL de l'artefact (même pattern que l'infographie). À tester en Étape 1.

8. Phase 3 — Gestion des erreurs

Le pipeline applique une stratégie de tolérance aux pannes avec `continue_on_error: true` par défaut.

Erreur	Stratégie	Action
Timeout polling (>10min)	Retry ×3 (30s entre chaque)	Log failure → continuer les autres types
download_artifact échoue	Fallback httpx+CDP si infographic, sinon log	Marquer type comme ☐ dans rapport final
Type non supporté par le compte	Skip immédiat	Log warning + continuer
Rate limit NotebookLM (429)	Attente 60s + retry	Max 3 retries, puis skip
studio_create → erreur 500	Retry ×2 avec délai exponentiel	5s → 15s → skip
artifact_id non trouvé	Re-crée le studio si < 2 tentatives	Évite les artifacts orphelins

Structure du rapport de session

```
{
  "session": "2026-06-04_mon-notebook",
  "duration_s": 245,
  "results": {
    "infographic": {"status": "☐", "path": "outputs/2026-06-04_infographic.png"},
    "audio": {"status": "☐", "path": "outputs/2026-06-04_audio.m4a"},
    "quiz": {"status": "☐", "error": "Timeout après 3 tentatives"},
    "report": {"status": "☐", "path": "outputs/2026-06-04_report.md"}
  }
}
```

9. Cas d'usage concrets

CAS 1 — PACK VEILLE HEBDOMADAIRE COMPLET

Config : `output_types: ["report", "audio", "infographic"]`

→ Briefing Doc (MD) + Podcast deep-dive (M4A) + Infographie résumé (PNG). Idéal pour une note de veille distribuée chaque lundi.

CAS 2 — MODULE DE FORMATION E-LEARNING

Config : `output_types: ["slide_deck", "quiz", "flashcards", "audio"]`

→ Slides PDF + Quiz HTML interactif + Flashcards révision + Podcast intro. Un notebook = un module complet LMS.

CAS 3 — DOSSIER CLIENT JURIDIQUE ENRICHI

Config : `output_types: ["report", "data_table", "mind_map"]`

→ Briefing Doc (synthèse juridique) + CSV données clés + Mind map chronologie. Exportable vers Word ou Excel directement.

CAS 4 — PRÉSENTATION EXÉCUTIVE

Config : `output_types: ["slide_deck", "report", "infographic"]`

→ Deck PDF + Note exécutive + Visuel d'accroche. Prêt pour réunion ou email.

CAS 5 — PODCAST BILINGUE

Config : `output_types: ["audio"]` × 2 runs (language: fr + en)

→ Deux fichiers M4A dans deux langues. Même contenu, deux audiences.

CAS 6 — DATASET D'ANALYSE

Config : `output_types: ["data_table", "report"]`

→ CSV structuré + rapport d'analyse. Import direct Notion/Airtable/Google Sheets.

CAS 7 — RÉVISION EXAMEN / CERTIFICATION

Config : `output_types: ["flashcards", "quiz", "mind_map"]`

→ Pack révision complet : fiches mémo HTML + QCM interactif + carte mentale des concepts.

CAS 8 — CONTENU RÉSEAUX SOCIAUX

Config : `output_types: ["infographic", "report"]`

→ Visuel carré (LinkedIn/Instagram) + texte pour la légende. Workflow créatif automatisé.

CAS 9 — PIPELINE PLANIFIÉ (TÂCHE SCHEDULÉE)

Config : `execution.mode: "sequential"` + CRON hebdomadaire

→ Chaque lundi à 7h, le pipeline génère automatiquement le pack de la semaine depuis le notebook de veille mis à jour.

CAS 10 — MULTI-NOTEBOOKS BATCH

Boucle sur plusieurs `notebook_id`, un `pipeline_config.json` par projet

→ Génère les sorties de 5 notebooks différents en une seule exécution. Sorties dans `outputs/{notebook_name}/`.

CAS 11 — ARCHIVAGE DOSSIER ADMINISTRATIF

Config : `output_types: ["report", "data_table", "slide_deck"]` + post-proc PDF

→ Tout le dossier converti en pack archivable : rapport PDF + Excel + présentation. Idéal fin de dossier client.

CAS 12 — DIFFUSION FAMILIALE (FAMILLE MAROCAINE)

Config : `output_types: ["audio", "infographic", "flashcards"]` + `language: "ar"`

→ Support pédagogique en arabe pour expliquer des démarches administratives complexes à distance.

10. Ordre d'implémentation recommandé

1 Étape 1 — Tests de download_artifact (priorité critique)

Tester live `download_artifact` pour audio (.m4a), slide_deck (.pdf) et mind_map (.json) sur Windows. Identifier les bugs potentiels de streaming binaire. Durée estimée : 1-2h.

2 Étape 2 — Extension pipeline_config.json + SCHEDULED_TASK.md

Ajouter le schéma v2 avec tous les types. Mettre à jour le script de tâche planifiée pour lire la nouvelle config.

3 Étape 3 — Développer le routeur multi-types (TYPE_ROUTER)

Implémenter `build_studio_params()` pour les 9 types. Ajouter la boucle principale avec artifacts_map.

4 Étape 4 — Post-processeurs

Mind map JSON → HTML D3.js. Report Markdown → PDF WeasyPrint. Fiche métadonnées pour audio/vidéo.

5 Étape 5 — Configurateur v2

Interface interactive (widget HTML ou CLI) pour sélectionner les types souhaités et générer le `pipeline_config.json` sans édition manuelle.

6 Étape 6 — Tests end-to-end

Exécuter le pipeline complet sur un notebook de test avec tous les types activés. Valider chaque sortie. Documenter les résultats.

7 Étape 7 — Mise à jour repo GitHub + CHANGELOG v2.0

Pousser la nouvelle version, taguer v2.0.0, mettre à jour le README avec les nouveaux types supportés.

Composant	Priorité	Complexité	Durée est.
pipeline_config.json schema v2	❑ Critique	Faible	30min
Routeur multi-types (TYPE_ROUTER)	❑ Critique	Moyenne	2-3h
Handler download audio/vidéo	❑ Critique	❑ Inconnue (streaming)	1-4h
Handler download slide_deck PDF	❑ Critique	❑ Inconnue	1-2h
Post-processor mind_map → HTML	❑ Utile	Faible (D3.js)	1h

Post-processor markdown → PDF	🟡 Utile	Faible (WeasyPrint)	30min
Configurateur v2	🟡 Utile	Moyenne	2h
Rapport de session multi-outputs	🟢 Nice	Faible	30min

11. Post-processeurs optionnels

Mind Map JSON → Visualisation HTML (D3.js)

```
def postprocess_mindmap(json_path: str, output_dir: str) -> str:
    """Génère une visualisation D3.js depuis le JSON mind_map de NLM."""
    with open(json_path) as f:
        data = json.load(f)
    html = D3_MINDMAP_TEMPLATE.format(data=json.dumps(data))
    html_path = json_path.replace(".json", "_viz.html")
    Path(html_path).write_text(html, encoding="utf-8")
    return html_path
```

Report Markdown → PDF WeasyPrint

```
def postprocess_report_to_pdf(md_path: str) -> str:
    """Convertit un rapport Markdown NLM en PDF brandkité."""
    import markdown
    html = markdown.markdown(Path(md_path).read_text())
    full_html = REPORT_PDF_TEMPLATE.format(content=html)
    pdf_path = md_path.replace(".md", ".pdf")
    subprocess.run(["python", "-m", "weasyprint",
                    "report.html", pdf_path], check=True)
    return pdf_path
```

Audio/Vidéo → Fiche métadonnées

```
def generate_media_metadata(media_path: str, artifact_info: dict) -> str:
    """Génère une fiche .txt avec les métadonnées de l'audio/vidéo."""
    meta = {
        "fichier": Path(media_path).name,
        "type": artifact_info.get("output_type"),
        "format": artifact_info.get("audio_format", "N/A"),
        "notebook_id": artifact_info.get("notebook_id"),
        "généré_le": datetime.now().isoformat(),
        "taille_mo": Path(media_path).stat().st_size / 1_048_576
    }
    txt_path = media_path.rsplit(".", 1)[0] + "_meta.txt"
    Path(txt_path).write_text(
        "\n".join(f"{k}: {v}" for k, v in meta.items())
    )
    return txt_path
```

12. Dépannage — Erreurs connues & solutions

Erreur	Cause	Solution
403 Forbidden sur lh3.googleusercontent.com	Cookie OSID présent (bug Windows CDP)	Supprimer OSID des cookies avant httpx → validé v1
download_artifact → fichier vide (0 bytes)	Streaming binaire coupé sur Windows	Utiliser handler httpx+CDP en fallback avec l'URL de l'artefact
studio_create → "type not supported"	Type non activé sur le compte NLM	Vérifier les types disponibles via l'interface NLM, retirer de output_types
Timeout polling après 10min	NLM surchargé ou notebook trop volumineux	Augmenter timeout_min à 20, réduire les sources du notebook
Chemin Windows avec backslash	Bug Python/MCP sur Windows paths	Toujours utiliser <code>pathlib.Path</code> ou <code>r"C:\..."</code>
Rate limit 429 de NLM	Trop de studio_create en parallèle	Passer en mode sequential + délai 30s entre appels
slide_deck → URL lh3.googleusercontent.com	Même problème que l'infographie	Utiliser handler httpx+CDP (supprimer OSID) si MCP échoue
quiz/flashcards → HTML invalide	Encodage UTF-8 manquant	Ouvrir avec <code>encoding="utf-8"</code> et valider avec BeautifulSoup

13. Artifact Configurateur v2 — Interface Cowork

L'artifact **Pipeline NLM Multi-Output v2** est un widget interactif intégré à Claude Desktop (Cowork). Il permet de configurer et lancer le pipeline entièrement via une interface graphique, sans éditer manuellement le fichier `pipeline_config_v2.json`.

Accès et démarrage

L'artifact est persistant — il survit entre les sessions Cowork. Pour l'ouvrir : cliquer sur **Pipeline NLM Multi-Output v2** dans le panneau d'artifacts Cowork. Au chargement, il interroge automatiquement `nlm notebook list --json` et affiche la liste de vos notebooks en temps réel.

Étape 1 — Sélection du notebook

Le champ de recherche filtre instantanément parmi vos notebooks. Cliquer sur un notebook le sélectionne : son titre, nombre de sources et ID s'affichent dans la barre de confirmation. Le bouton **Live**  force un rechargement depuis NotebookLM (utile si vous avez ajouté des notebooks depuis la dernière ouverture).

Étape 2 — Sélection des types de sortie

Une grille de 9 cartes permet de cocher indépendamment chaque format souhaité :

Icône	Type	Format généré	Cas d'usage principal
☐	Audio	.m4a	Podcast, écoute en déplacement
☐	Vidéo	.mp4	Présentation animée
☐	Slides	.pdf / .pptx	Présentation, réunion
☐	Infographie	.png	Synthèse visuelle, réseaux sociaux
☐	Rapport	.md	Briefing, note exécutive
☐	Mind Map	.json	Cartographie conceptuelle
☒	Flashcards	.html	Révision, mémorisation
☐	Quiz	.html	Évaluation, formation
☐	Tableau	.csv	Import Excel/Sheets/Notion

Les boutons **Tout sélectionner** et **Tout désélectionner** permettent une sélection rapide. Le badge bleu affiche le nombre de types actifs.

Étape 3 — Options par type

Pour chaque type coché, un panneau d'options s'ouvre automatiquement à l'étape 3. Les options disponibles varient selon le type :

☒ **Audio** — Format (Deep Dive / Bref / Aperçu / Critique / Débat), Durée (Court / Standard / Long), Langue (FR/EN/AR)

☒ **Slides** — Format (Détaillé / Présentateur), Fichier (PDF / PPTX), Langue

☒ **Infographie** — Orientation (Paysage / Portrait / Carré), Style (Auto / Pâte / Croquis / Animé / Éditorial / Éducatif / Bento / Briques / Scientifique), Langue

☒ **Rapport** — Type (Briefing Doc / Study Guide / FAQ / Timeline), Langue

☒ **Quiz** — Nombre de questions (5 / 10 / 15 / 20), Niveau (Facile / Moyen / Difficile)

☒ **Flashcards** — Niveau de difficulté

☒ **Mind Map** — Titre personnalisé

Étape 4 — Paramètres d'exécution

Paramètre	Valeurs	Recommandation
-----------	---------	----------------

Mode d'exécution	Séquentiel / Parallèle	Séquentiel (évite rate limits NLM)
Continuer si erreur	Oui / Non	Oui (passe au type suivant)
Intervalle polling	10 / 15 / 20 / 30s	15s (par défaut)
Timeout par type	8 / 12 / 20 / 30 min	12min (audio peut prendre ~8min)
Dossier de sortie	Chemin Windows	pipelines/notebooklm-infographier/outputs

Étape 5 — Sauvegarder & Lancer

Un récapitulatif confirme le notebook sélectionné, les types actifs et le mode d'exécution. Deux actions disponibles :

 **Sauvegarder** — Écrit `pipeline_config_v2.json` dans le dossier pipeline. Le fichier peut être réutilisé directement en ligne de commande avec `python pipeline_runner_v2.py --config pipeline_config_v2.json`.

 **Lancer** — Sauvegarde la config puis démarre `pipeline_runner_v2.py` en arrière-plan via PowerShell. Un journal de logs en temps réel s'affiche dans l'artifact : statuts de création, polling, downloads et rapport final.

⚠ **Prérequis au lancement** : Comet (Perplexity) doit être installé sur le système. L'artifact détecte automatiquement si Comet tourne — si ce n'est pas le cas, `pipeline_runner_v2.py` le lance en arrière-plan, navigue vers NotebookLM et extrait les cookies Google automatiquement.

Workflow type — de l'artifact au fichier

```
Ouvrir artifact v2
→ Sélectionner notebook (ex: "Affaire Rakuten")
→ Cocher : audio + slide_deck + report + infographic
→ Options : audio FR deep_dive · slides PDF FR · infographie paysage auto
→ Exécution : séquentiel · 15s · 12min
→ Clic ► Lancer
    ↓
pipeline_config_v2.json sauvegardé
pipeline_runner_v2.py lancé (fond)
    ↓
[Log] ✓ Auth Comet OK — 2391 cookies
[Log] ✓ Notebook résolu : Affaire Rakuten (63 sources)
[Log] ✓ audio completed en 482s — 30 MB
[Log] ✓ slide_deck completed en 95s — 8.9 MB
[Log] ✓ report completed en 28s — 6 KB
[Log] ✓ infographic completed en 67s — 2.4 MB
[Log] === TOUS LES TYPES GÉNÉRÉS ◻ ===
```

14. Ressources & Références

Ressource	URL / Localisation
NotebookLM Studio	https://notebooklm.google.com
Documentation MCP NLM	Voir fichier SCHEDULED_TASK.md du repo
Pipeline v1 (infographie)	Repo GitHub → branch main → pipeline_v1/
Script download_infographie.py	Repo GitHub → scripts/download_infographie.py
pipeline_config.json template	Repo GitHub → config/pipeline_config_template.json
Chrome DevTools Protocol (CDP)	https://chromedevtools.github.io/devtools-protocol/
WeasyPrint	https://weasyprint.org · pip install weasyprint
D3.js (mind map)	https://d3js.org
PDF_STANDARDS.md	%AppData%\Claude\standards\PDF_STANDARDS.md

❑ **Prochaine session recommandée** : Démarrer par l'Étape 1 — tester live `download_artifact` pour audio (.m4a) et slide_deck (.pdf) sur votre système Windows. Cela débloquera la certitude de 70% → 100% sur ces types avant de coder le pipeline complet.